



1

자료구조 개요

IT CookBook, 자바로 배우는 쉬운 자료구조



1

자료구조 개요

2

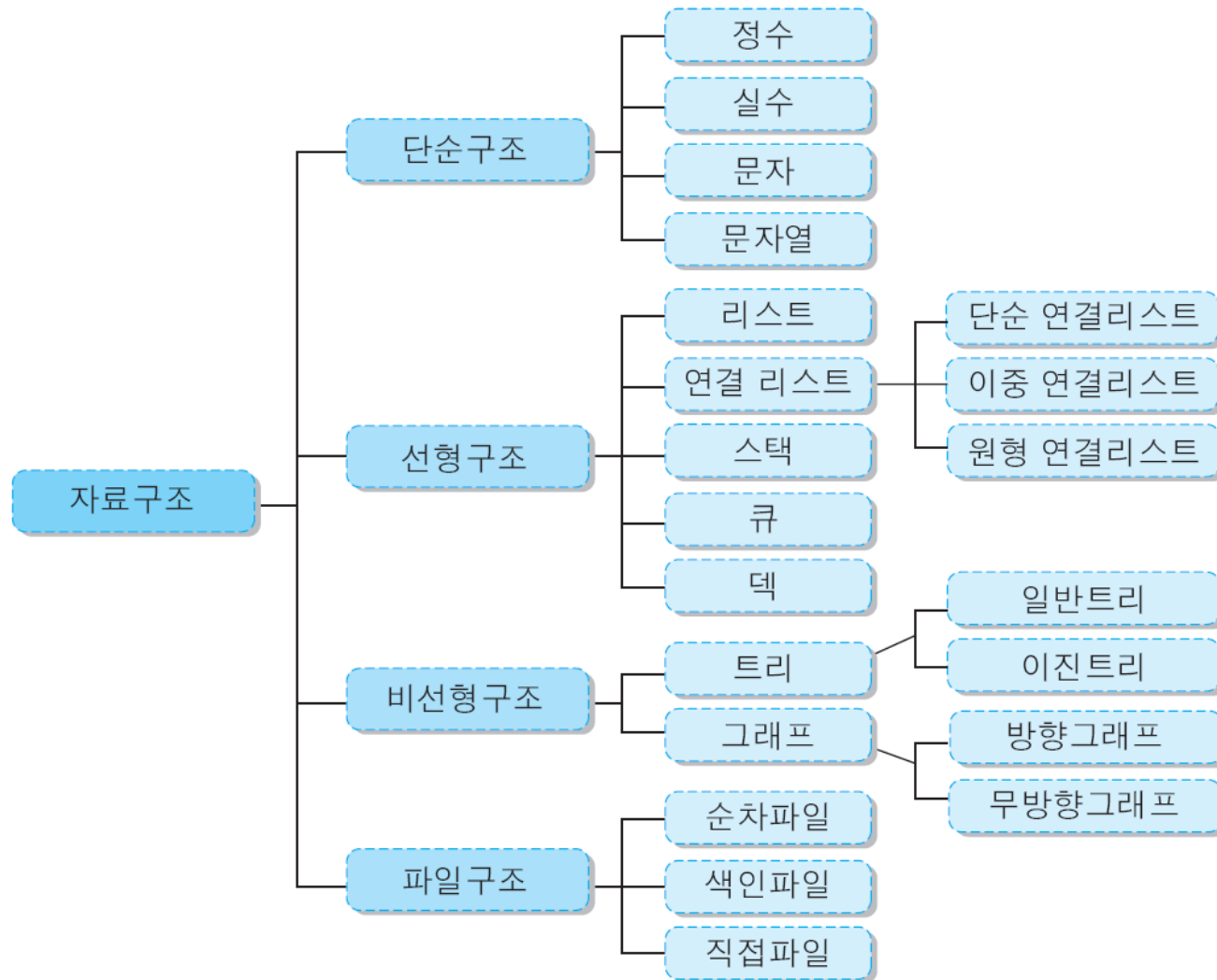
자료구조의 분류

3

자료의 표현

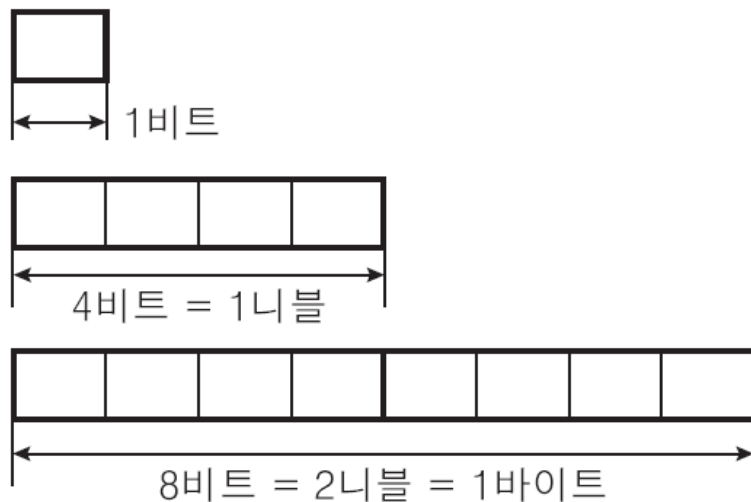
❖ 자료구조란?

- 자료를 효율적으로 사용하기 위해서 자료의 특성에 따라서 분류하여 구성하고 저장 및 처리하는 모든 작업
- 데이터형: 시스템 제공 데이터 형과 사용자 정의 데이터 형 존재
 - ✓ 시스템 제공 데이터 형: int, char, float, double 등
 - ✓ 사용자 정의 데이터 형: 시스템 제공 데이터 형을 결합하여 새로운 데이터 형을 만듦
- 배열, 리스트, 스택 등이 자료구조의 대표적 방법론이다.
- ADT(Abstract Data Type): 시스템 제공 데이터 형은 시스템에서 Data의 연산(사칙 연산, 함수 등) 방법을 제공한다.
하지만, 사용자 정의 데이터 형은 사용자가 연산과정을 만들어야 하는데 ADT는 데이터 형과 연산을 결합한 것을 ADT라고 한다.
- ADT에서 구체적 알고리즘은 표현하지 않는다.



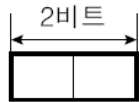
❖ 디지털 시스템에서의 자료의 표현

- 숫자, 문자, 그림, 소리, 기호 등 모든 형식의 자료를 2진수 코드로 표현하여 저장 및 처리
- 2진수 코드
 - 1과 0, ON과 OFF, 참(True)과 거짓(False)의 조합
- 2진수 코드의 단위



■ n개의 비트로 2^n 개의 상태수 표현

• 예) $n=2$ 인 경우



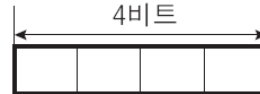
$n=2$



0	0
0	1
1	0
1	1

4개 ($2^n = 2^2 = 4$)의 상태값 표현

예) $n=4$ 인 경우



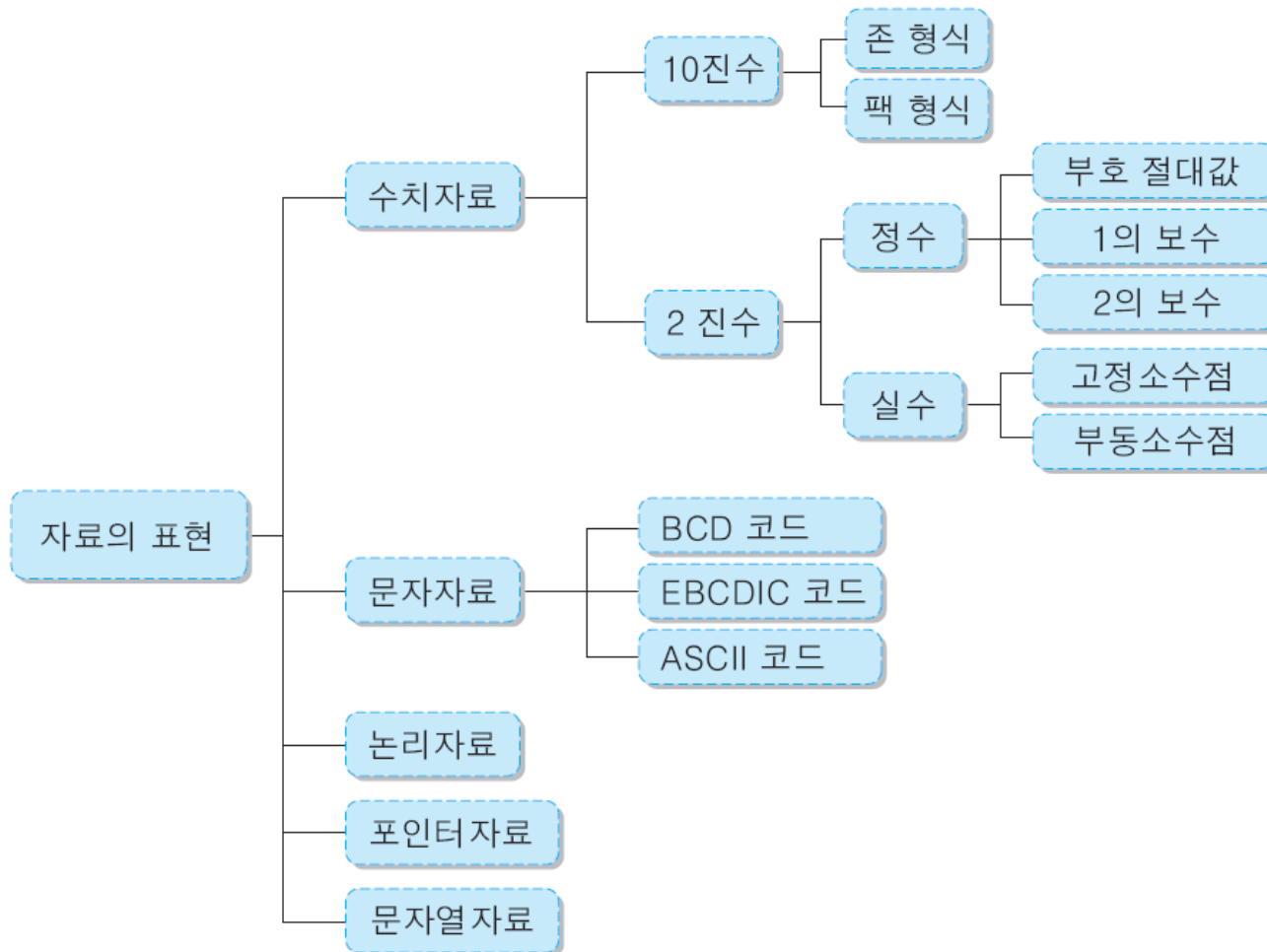
$n=4$



0	0	0	0
0	0	0	1
0	0	1	0
0	0	1	1
0	1	0	0
0	1	0	1
0	1	1	0
0	1	1	1
1	0	0	0
1	0	0	1
1	0	1	0
1	0	1	1
1	1	0	0
1	1	0	1
1	1	1	0
1	1	1	1

16개 ($2^n = 2^4 = 16$)의 상태값 표현

❖ 컴퓨터 내부에서 표현할 수 있는 자료의 종류



❖ 2진수의 정수 표현

■ n비트의 부호 절대값 형식

- 최상위 1비트 - 부호 표시
 - 양수(+) : 0
 - 음수(-) : 1
- 나머지 n-1 비트 - 이진수 표시
- 1바이트를 사용하는 부호 절대값 형식의 예
 - + 21
 - - 21

1비트	← 7 비트 →
0	0 0 1 0 1 0 1
부호	절대값 = 21

1비트	← 7 비트 →
1	0 0 1 0 1 0 1
부호	절대값 = 21

■ 2의 보수(2' Complement) 형식

2진수	십진수	2진수	십진수
0000	+0	1000	-8
0001	1	1001	-7
0010	2	1010	-6
0011	3	1011	-5
0100	4	1100	-4
0101	5	1101	-3
0110	6	1110	-2
0111	7	1111	-1

➤ $2+3 \rightarrow 0010 + 0011 = 0101$

➤ $-2-3 \rightarrow 1110 + 1101 = 1011$

➤ $3-2 \rightarrow 0011 + 1110 = 0001$

➤ $2-3 \rightarrow 0010 + 1101 = 1111$

❖ 2진수의 실수 표현

■ 고정 소수점 표현

- 정수부와 소수부로 표현함
- 12.34는 정수부에 12, 소수부에 34를 기억
- 32비트라면 부호비트를 제외한 31비트에 정수부와 소수부를 나누어 저장

■ 부동 소수점 형식의 표현

- 고정 소수점 형식에 비해서 표현 가능한 값의 범위가 넓다.
- 실수를 부호와 지수, 소수의 세 부분으로 구분하여 표현

$$213 = \underbrace{0.213}_{\text{소수부}} \times \underbrace{10^3}_{\text{밑수 (base, radix)}} \longrightarrow \text{지수}$$



❖ 문자자료의 표현

- 문자에 대한 이진수 코드를 정의하여 사용
- 문자에 대한 이진수 코드표
 - BCD 코드
 - EBCDIC 코드
 - ASCII 코드

❖ BCD 코드

■ 6비트를 사용하여 문자 표현

- 상위 2비트 : 존 비트
- 하위 4비트 : 2진수 비트
- 존 비트와 2진수 비트를 조합하여 10진수 0~9와 영어 대문자와 특수문자를 표현

■ BCD 코드의 구성

존 비트		← 숫자 비트 →			
A	B	8	4	2	1
x	x	x	x	x	x

존 비트 AB의 값

00	: 0, 1~9(1010, 0001~1001)
01	: 문자 A~I(0001~1001)
10	: 문자 J~R(0001~1001)
11	: 문자 S~Z(0010~1001)

■ BCD 코드 표

존 비트		숫자 비트				표현 문자	존 비트		숫자 비트				표현 문자	존 비트		숫자 비트				표현 문자	존 비트		숫자 비트				표현 문자
0	0	0	0	0	1	1	0	1	0	0	0	1	A	1	0	0	0	0	1	J							
0	0	0	0	1	0	2	0	1	0	0	1	0	B	1	0	0	0	1	0	K	1	1	0	0	1	0	S
0	0	0	0	1	1	3	0	1	0	0	1	1	C	1	0	0	0	1	1	L	1	1	0	0	1	1	T
0	0	0	1	0	0	4	0	1	0	1	0	0	D	1	0	0	1	0	0	M	1	1	0	1	0	0	U
0	0	0	1	0	1	5	0	1	0	1	0	1	E	1	0	0	1	0	1	N	1	1	0	1	0	1	V
0	0	0	1	1	0	6	0	1	0	1	1	0	F	1	0	0	1	1	0	O	1	1	0	1	1	0	W
0	0	0	1	1	1	7	0	1	0	1	1	1	G	1	0	0	1	1	1	P	1	1	0	1	1	1	X
0	0	1	0	0	0	8	0	1	1	0	0	0	H	1	0	1	0	0	0	Q	1	1	1	0	0	0	Y
0	0	1	0	0	1	9	0	1	1	0	0	1	I	1	0	1	0	0	1	R	1	1	1	0	0	1	Z
0	0	1	0	1	0	0																					

- 예) 영문자 A에 대한 BCD 코드 ➡ 010001

❖ EBCDIC 코드

■ 8비트를 사용하여 문자 표현

- 상위 4비트 : 존 비트
- 하위 4비트 : 2진수 비트
- 존 비트와 2진수 비트를 조합하여 10진수 0~9와 영어 대문자/소문자와 특수문자를 표현

■ EBCDIC 코드의 구성

← 존 비트 →				← 숫자 비트 →			
A	B	C	D	8	4	2	1
x	x	x	x	x	x	x	x

존 비트 AB의 값

- 00 : 여분
- 01 : 특수 문자
- 10 : 영어 소문자
- 11 : 영어 대문자

존 비트 CD의 값

- 00 : 문자 A~I(0001~1001)
- 01 : 문자 J~R(0011~1011)
- 10 : 문자 S~Z(0010~1001)
- 11 : 숫자 0~9(0000~1001)

■ EBCDIC 코드 표

상위 하위	0000	0001	0010	0011	0100	0101	0110	0111	1000	1001	1010	1011	1100	1101	1110	1111
0000	NUL	DLE	DS		SP	&	-						{	}	₩(\)	0
0001	SOH	DC1	SOS			/			a	j	~		A	J		1
0010	STX	DC2	FS	SYN					b	k	s		B	K	S	2
0011	ETX	TM							c	l	t		C	L	T	3
0100	PF	RES	BYP	PN					d	m	u		D	M	U	4
0101	HT	NL	LF	RS					e	n	v		E	N	V	5
0110	LC	BS	ETB	UC					f	o	w		F	O	W	6
0111	DEL	IL	ESC	EOT					g	p	x		G	P	X	7
1000	GE	CAN							h	q	y		H	Q	Y	8
1001	RLF	EM							i	r	z		I	R	Z	9
1010	SMM	CC	SM		∅	!	:									
1011	VT	CU1	CU2	CU3	.	\$	,	#								
1100	FF	IFS		DC4	<	*	%	@								
1101	CR	IGS	ENQ	NAK	(	)	-	'								
1110	SO	IRS	ACK		+	;	>	=								
1111	SI	IUS	BEL	SUB		┌	?	"								

- 예) 영문자 A에 대한 EBCDIC 코드 ➡ 11000001

❖ ASCII 코드

■ 7비트를 사용하여 문자 표현

- 상위 3비트 : 존 비트
- 하위 4비트 : 2진수 비트
- 존 비트와 2진수 비트를 조합하여 10진수 0~9와 영어 대문자/소문자와 특수문자를 표현

■ ASCII 코드의 구성

← 존 비트 →			← 숫자 비트 →			
			8	4	2	1
x	x	x	x	x	x	x

■ ASCII 코드 표

하위 \ 상위	000	001	010	011	100	101	110	111
0000	NUL	DLE	SP	0	@	P	`	p
0001	SOH	DC1	!	1	A	Q	a	q
0010	STX	DC2	"	2	B	R	b	r
0011	ETX	DC3	#	3	C	S	c	s
0100	EOT	DC4	\$	4	D	T	d	t
0101	END	NAK	%	5	E	U	e	u
0110	ACK	SYN	&	6	F	V	f	v
0111	BEL	ETB	'	7	G	W	g	w
1000	BS	CAN	(	8	H	X	h	x
1001	HT	EM	)	9	I	Y	i	y
1010	LF	SUB	*	:	J	Z	j	z
1011	VT	ESC	+	;	K	[	k	{
1100	FF	FS	,	<	L	₩(\)	l	
1101	CR	GS	-	=	M	]	m	}
1110	SO	RS	.	>	N	^	n	~
1111	SI	US	/	?	O	_	o	DEL

※ 코드의 의미

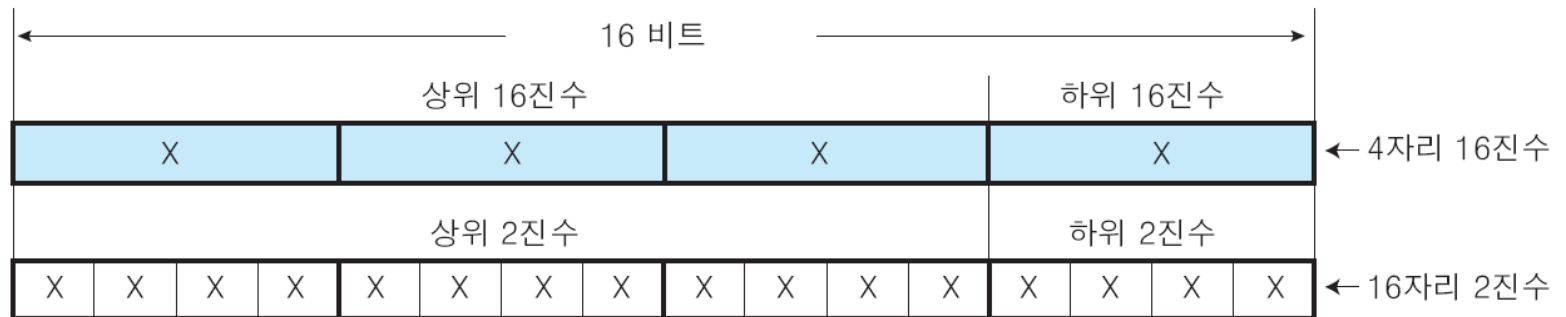
GS	Group Separator	RS	Record Separator	US	Unit Separator
----	-----------------	----	------------------	----	----------------

- 예) 영문자 A에 대한 ASCII 코드 ➡ 1000001=65로 1바이트에 표현가능

❖ 유니 코드

■ 16비트를 사용하여 문자 표현

- 16비트의 코드값을 4자리의 16진수로 표시
- 유니 코드의 구성



■ 유니 코드 표

상위16진수 하위16진수	000	001	002	003	004	005	006	007
0	NUL	DEL	SP	0	@	P	`	p
1	SOH	DC1	!	1	A	Q	a	q
2	STX	DC2	"	2	B	R	b	r
3	ETX	DC3	#	3	C	S	c	s
4	EOT	DC4	\$	4	D	T	d	t
5	ENQ	NAK	%	5	E	U	e	u
6	ACK	SYN	&	6	F	V	f	v
7	BEL	ETB	'	7	G	W	g	w
8	BS	CAN	(	8	H	X	h	x
9	HT	EM	)	9	I	Y	i	y
A	LF	SUB	*	:	J	Z	j	z
B	VT	ESC	+	;	K	[	k	{
C	FF	FS	,	<	L	\	l	

D	CR	GS	-	=	M	]	m	}
E	SO	RS	.	>	N	^	n	~
F	SI	US	/	?	O	_	o	DEL

- 영문자 A에 대한 유니 코드 ➡ 0041 ➡ 0000 0000 0100 0001

← 16 비트 →															
상위 16진수												하위 16진수			
0				0				4				1			
상위 2진수												하위 2진수			
0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	1

- 한글에 대한 유니코드

➤ 초성 19개 × 중성 21개 × 종성 28개 = 11,172개

[표 1-6] 유니코드표: 한글에 대한 유니코드 중 일부

상위16진수 하위16진수	AC0	AC1	AC2	AC3	AC4	AC5	AC6	AC7
0	가	감	갸	갹	갈	각	갸	거
1	각	갑	갸	갹	갈	갈	갸	걱
2	갸	갸	갸	갹	갈	갸	갸	겨
3	갸	갸	갸	갹	갈	갸	갸	깁
4	간	갸	갸	갹	갈	개	갸	건
5	갸	강	갸	갸	갈	객	갸	깁
6	갸	갸	갸	갹	갈	객	갸	깁
7	간	갸	갸	갹	갈	갸	갸	깁
8	갈	각	갸	가	감	갸	갸	겉
9	갸	갈	갸	각	갑	갸	갸	겉
A	갸	갸	갸	각	갑	갸	갸	겉
B	갸	갸	갸	갸	갸	갸	갸	겉
C	갸	개	갸	갸	갸	갸	갸	겉
D	갸	객	갸	갸	강	갸	갸	겉
E	갸	객	갸	갸	갸	갸	갸	겉
F	갸	갸	갸	갸	갸	갸	갸	겉

- 한글 '가'에 대한 유니 코드 ➡ AC00 ➡ 1010 1100 0000 0000

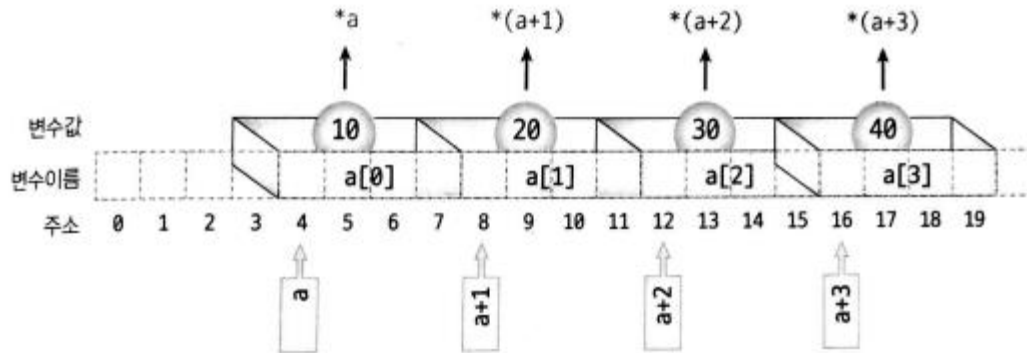
← 16 비트 →															
상위 16진수												하위 16진수			
A				C				0				0			
상위 2진수												하위 2진수			
1	0	1	0	1	1	0	0	0	0	0	0	0	0	0	0

❖ 논리자료

- 논리값을 표현하기 위한 자료 형식
- 논리값
 - 참(true)과 거짓(false), 1과 0
- 1바이트를 사용하여 논리자료를 표현하는 방법
 - 방법1)
 - 참 - 최하위 비트를 1로 표시. 00000001
 - 거짓 - 전체 비트를 0으로 표시. 00000000
 - 방법2)
 - 참 - 전체 비트를 1로 표시. 11111111
 - 거짓 - 전체 비트를 0으로 표시. 00000000
 - 방법3)
 - 참 - 하나 이상의 비트를 1로 표시
 - 거짓 - 전체 비트를 0으로 표시. 00000000

❖ 배열 자료

- 같은 타입의 자료형이 연속 공간에 저장된 형태



❖ 포인터 자료

- 메모리의 주소를 표현하기 위한 자료 형식
- `a`는 `a[0]`의 주소가 저장되어 있으며, `*a`는 해당 주소에 저장된 값으로 10이 출력됨
- `*(a+1)`은 그 다음 값을 의미함
- 이처럼 포인터는 변수의 주소나 메모리의 특정 위치에 대한 주소를 저장하고 주소연산하기 위해 사용

❖ 문자열(string) 자료

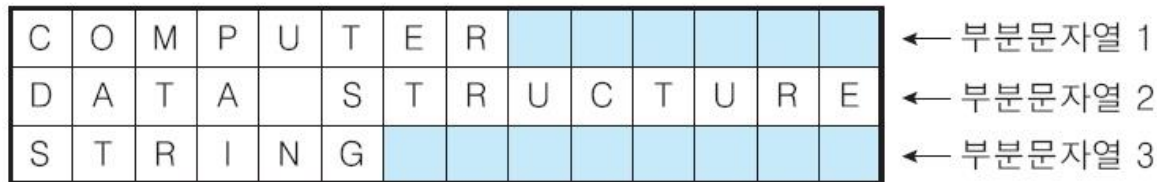
- 여러 문자로 이루어진 문자의 그룹을 하나의 자료로 취급하여 메모리에 연속적으로 저장하는 자료 형식
- 하나의 문자열 자료에 포함된 부분문자열을 표현하는 방법
 - 방법1) 부분문자열 사이에 구분자를 두고 연속 저장하는 방법
 - 방법2) 가장 긴 부분문자열의 길이에 맞추어 고정 길이로 연속 저장하는 방법
 - 방법3) 부분문자열을 연속 저장하고 각 부분문자열에 대한 포인터를 사용하는 방법

■ 부분문자열의 표현 예

- 표현할 문자열 : {COMPUTER, DATA STRUCTURE, STRING}
- 방법1) 구분자를 사용하는 표현 : 구분자로 세미콜론(;) 사용

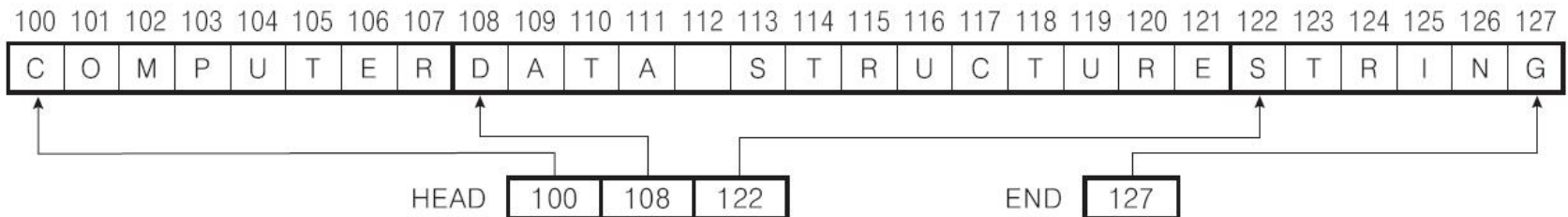


방법2) 고정길이를 사용하는 표현



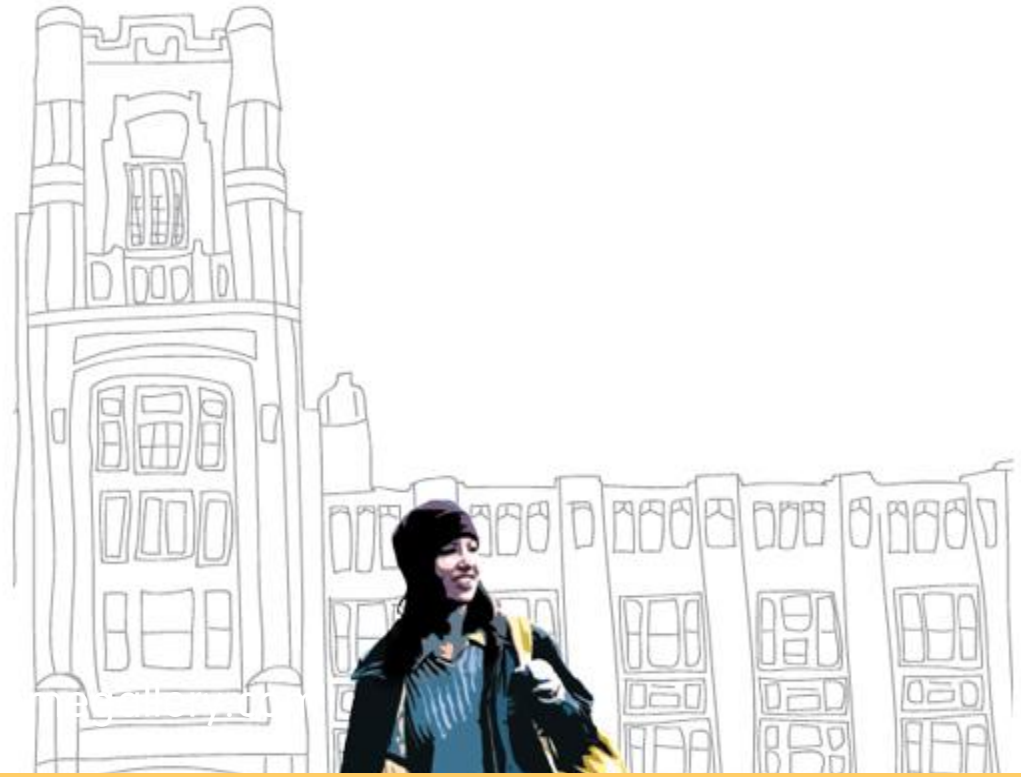
• 방법3) 포인터를 사용하는 표현

메모리 주소



■ 부분 문자열 표현 방법의 비교

비교항목 \ 방법	구분자를 사용하는 방법	고정길이를 저장하는 방법	포인터를 사용하는 방법
메모리 이용률	문자열 길이 + 구분자 길이 ☞ 효율적	가장 긴 부분문자열 길이 × 부분문자열의 개수 ☞ 비효율적	문자열 길이 + 포인터 저장공간 ☞ 효율적
부분문자열 탐색시간	문자 비교연산시간 + 구분자 식별시간 ☞ 비효율적	문자 비교연산시간 ☞ 효율적	문자 비교연산시간 + 포인터 주소연산시간 ☞ 효율적



Thank You !

IT CookBook 자바로 배우는 쉬운 자료구조 1장 끝

